

How Tokenization Works

Week 2 · Andrej Karpathy's 'Intro to LLMs' · Key Concept #1

Core idea: LLMs don't read characters or words — they read **tokens**. A token is a chunk of text (usually a subword) assigned a number from the model's vocabulary. Everything you type is converted to numbers before the model sees it.

1 · What Exactly Is a Token?

A token is the **smallest unit of text the model processes**. It is not a word, not a character — it sits in between. Think of it as a "chunk" that appears very frequently in text. The model converts every token into a number (its ID in the vocabulary), and that number is what actually gets fed into the neural network.

Text	Tokens (split)	# of tokens	Why?
cat	"cat"	1	Very common word → own token
playing	"pla" + "y" + "ing"	3	"ing" is common; stem split
xylophone	"xy" + "lo" + "phone"	3	Rare word → broken up
unbelievable	"un" + "bel" + "iev" + "able"	4	Prefix + stem + suffix chunks
Hello, world!	"Hello" + "," + " world" + "!"	4	Punctuation = separate tokens

2 · The BPE Process — Step by Step

BPE Tokenization Process — word 'playing' as example



BPE (Byte Pair Encoding) starts from scratch with individual characters and asks: *"What two chunks appear next to each other most often in our training text?"* It merges that pair into one new token and repeats -- hundreds of thousands of times -- until the vocabulary reaches its target size (~50k-100k tokens).

BPE in action — learning the token 'ing'

Step	Action	Result
Start	All text split into individual characters	i n g (separate)
Round 1	"i"+"n" appear together constantly (in, ing, finding...) → merge	in g
Round 2	"in"+"g" is extremely common (running, jumping, eating...) → merge	ing ← new token!
Result	"ing" is now a single token with its own ID (e.g., ID #4131)	'playing' = 'pla' + 'y' + 'ing'

What's Inside a ~100,000 Token Vocabulary

Common words 'the','is','cat'	Word stems 'play','run','talk'	Suffixes 'ing','ed','tion'	Prefixes 'un','re','pre'	Punctuation & numbers	Rare / foreign word pieces
----------------------------------	-----------------------------------	-------------------------------	-----------------------------	--------------------------	-------------------------------

Approximate breakdown — GPT-4 has ~100,328 tokens. Grey = rare fragments like 'xyl', 'phon'.

What "Predicting the Next Token" Actually Means

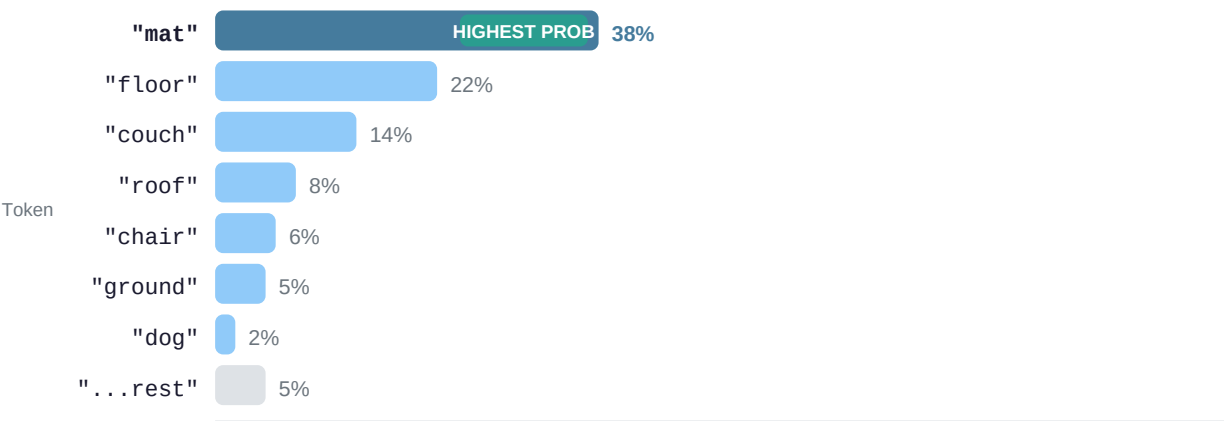
The model's only job — output a probability score for every token in its vocabulary.

Common misconception: "The model picks the next word."
Reality: The model outputs a probability distribution over ~100,000 tokens. Then a *separate sampling step* picks one based on those probabilities. This is why GPT gives different answers to the same question each time.

3 · The Probability Scorecard

When you type "The cat sat on the", the model runs billions of calculations and produces one number per token in its entire vocabulary. These numbers (converted to percentages) tell you how likely each token is to come next:

Model output after: "The cat sat on the ____"



All ~100,000 probabilities sum to exactly 100%. The model does NOT just pick 'mat' — it produces this entire scorecard.

The key insight: the model doesn't "know" what comes next — it calculates *relative likelihood*. Most of the ~100,000 tokens get near-zero probability ("moon", "justice", "XYZ" are very unlikely after "The cat sat on the"). A handful get most of the probability mass. Then one is sampled.

4 · From Scorecard to Actual Word — Sampling

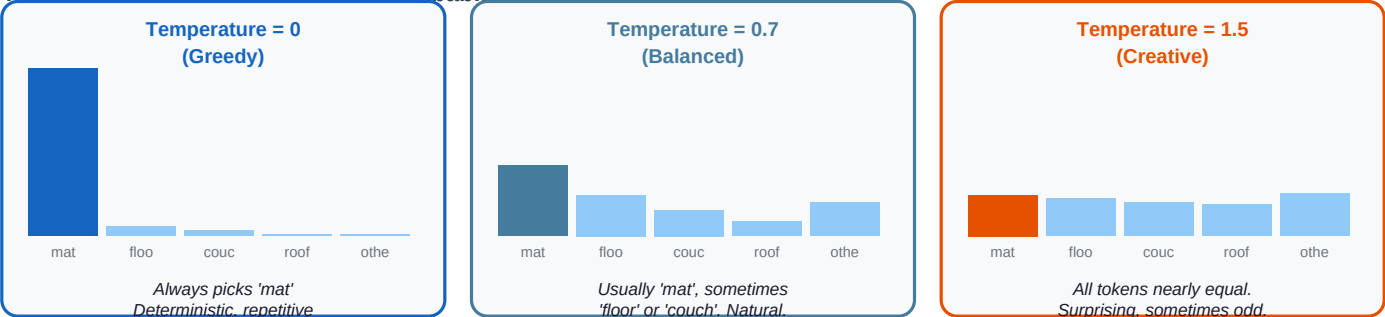
Strategy	How it works	Result	Used for
Greedy (temp = 0)	Always pick the single highest-probability token	Always 'mat '	Deterministic tasks, code completion
Sampling (temp = 0.7)	Treat probabilities as a weighted lottery — 'mat' wins 38% of the time, 'floor' 22%, etc.	Usually 'mat ', sometimes 'floor '	Chat, writing — natural variation

Top-k / Top-p	Sample only from the top k tokens (or tokens covering top p% of prob mass). Cuts off unlikely tokens.	Reasonable variation, no nonsense	Most production LLM APIs
----------------------	---	-----------------------------------	--------------------------

5 · Temperature — Sharpening or Flattening the Distribution

Temperature is a number (usually 0–2) that reshapes the probability distribution before sampling. Low temperature makes the model more confident and repetitive. High temperature makes it more creative — and sometimes weird.

Temperature: How It Changes the Probability Landscape



← More predictable / focused More creative / random →

6 · The Full Generation Loop

Every word you see in a ChatGPT response is produced by repeating this loop:

Step	What happens	Example
1	Your input text is tokenized into a list of IDs	"The cat sat on the" → [464, 3797, 3332, 319, 262]
2	IDs passed through the neural network (billions of matrix multiplications)	Forward pass through transformer layers
3	Network outputs one probability score per vocabulary token (~100k numbers)	mat=38%, floor=22%, couch=14%, ...
4	One token is sampled from the distribution based on temperature setting	"mat" selected (token ID 6749)
5	Selected token appended to input; loop repeats from step 1	Input is now: "The cat sat on the mat"
Repeat	This loop runs once per output token until [END] token is generated	Typical response = 100–500 loop iterations

Token Gotchas & Key Takeaways

Knowing why tokens behave unexpectedly makes you a better AI engineer.

7 · Surprising Token Behaviours (Common Interview Topics)

Gotcha	Why it happens	Example
LLMs are bad at counting letters	"strawberry" might be 3 tokens: "str" + "awb" + "erry". The model never sees individual letters — it sees token IDs.	Q: How many r's in "strawberry"? A: Often wrong!
Arithmetic is hard	"1234567" might be one token or several depending on length. Numbers don't map cleanly to tokens, so addition/subtraction requires workarounds.	"99 + 1" is easier than "99999 + 1" for the model
Spaces are part of tokens	The token for " cat" (with leading space) is different from "cat" (no space). This matters for prompting and parsing.	" playing" ≠ "playing" in token space
Different languages, different efficiency	English text uses ~1 token per word. Chinese/Arabic/Hindi often use more tokens per character since the vocab was built mostly on English text.	English: 10 words ≈ 10 tokens Chinese: 10 characters ≈ 15–20 tokens
Tokens affect cost	LLM APIs charge per token (input + output). A longer prompt = more tokens = higher cost. Token efficiency matters in production.	GPT-4: ~\$0.01 per 1k tokens (pricing varies by model)

8 · Key Takeaways — What You Now Know

Tokens are subword chunks, not words

BPE builds a vocabulary of ~50k–100k tokens. Common words get 1 token. Rare words are split into pieces. Every chunk has a unique numeric ID.

Prediction = probability distribution, not a single answer

The model outputs a score for every token in its vocabulary simultaneously. The highest score wins (or is sampled from). This is why the model can be "creative" — it doesn't always pick the most likely token.

Temperature controls certainty vs. creativity

Temp = 0 → always pick the top token (deterministic). Temp = 1 → sample proportionally. Temp > 1 → flatten the distribution, increase randomness.

One forward pass produces one token

Generating a 100-word response means running the entire neural network 100+ times, appending one token each time. Each token 'sees' all previous tokens.

The simple objective → general intelligence

The model was trained on only one task: predict the next token accurately. Nothing else. All reasoning, knowledge, and language skill emerged as a side effect of doing this extremely well across trillions of tokens of text.

Task Checklist — Week 2 · Karpathy Tokens

■	Watch Andrej Karpathy's 'Intro to LLMs' (youtube.com/watch?v=zjkBMFhNj_g)	~1 hour
■	Can explain what a token is in 1 sentence (not "word", not "character")	Self-test
■	Can describe BPE in simple steps (start with chars → merge frequent pairs)	Self-test
■	Understand that prediction = full probability distribution over ~100k tokens	Key concept
■	Know what temperature does (sharpens or flattens the distribution)	Key concept
■	Can explain why LLMs are bad at counting letters (tokens, not characters)	Gotcha
■	Tried the OpenAI tokenizer tool: platform.openai.com/tokenizer	Hands-on